

# **CASE STUDY ANALYSIS**

## **AI-BASED FLOOD DETECTION AND EARLY WARNING SYSTEM**

CSA1017- Software Engineering

Ragul SG - 192525007

# TABLE OF CONTENTS

---

|                                                                          |    |
|--------------------------------------------------------------------------|----|
| TABLE OF CONTENTS.....                                                   | 2  |
| 1. Understanding the Problem Statement .....                             | 4  |
| 1.1 Problem Statement in Own Words.....                                  | 4  |
| 1.2 Objectives of the Proposed System.....                               | 4  |
| 1.3 Scope of the Problem.....                                            | 5  |
| 2. Key Stakeholders and Challenges .....                                 | 6  |
| 2.1 Stakeholder Identification, Roles and Responsibilities .....         | 6  |
| 2.2 Major Challenges and Issues Faced by Stakeholders.....               | 7  |
| 2.2.1 Data-Related Challenges .....                                      | 7  |
| 2.2.2 Operational and Organisational Challenges.....                     | 8  |
| 2.2.3 Technical Challenges .....                                         | 8  |
| 3. Analysis of the Current Approach.....                                 | 9  |
| 3.1 Description of the Existing System / Current Process.....            | 9  |
| 3.2 Strengths of the Current System.....                                 | 9  |
| 3.3 Limitations of the Current System.....                               | 10 |
| 3.4 Gaps That Need to Be Addressed .....                                 | 10 |
| 4. Proposed Software Solution Using Software Engineering Principles..... | 11 |
| 4.1 Overview of the Proposed System.....                                 | 11 |
| 4.2 Major Functional Modules .....                                       | 11 |
| 4.3 Functional Requirements .....                                        | 12 |
| 4.4 Non-Functional Requirements .....                                    | 13 |
| 4.5 Incorporation of Software Engineering Principles .....               | 14 |
| 4.5.1 Modularity .....                                                   | 14 |
| 4.5.2 Scalability .....                                                  | 14 |
| 4.5.3 Maintainability.....                                               | 14 |
| 4.5.4 Reliability .....                                                  | 14 |

|                                                                                    |    |
|------------------------------------------------------------------------------------|----|
| 4.5.5 Usability .....                                                              | 15 |
| 4.5.6 Security.....                                                                | 15 |
| 4.6 High-Level Architecture Summary .....                                          | 15 |
| 5. Justification of the Chosen Methodology/Framework.....                          | 16 |
| 5.1 Selected Methodology: Agile (Scrum) combined with DevOps Practices.....        | 16 |
| 5.2 Comparison of Candidate Methodologies .....                                    | 16 |
| 5.3 Justification for Selecting Agile Scrum.....                                   | 17 |
| 5.4 How the Chosen Framework Supports Successful Development and Maintenance ..... | 17 |
| 6. Expected Outcomes and Limitations .....                                         | 19 |
| 6.1 Expected Benefits of the Proposed Solution.....                                | 19 |
| 6.2 How the Solution Addresses the Identified Challenges .....                     | 19 |
| 6.3 Potential Risks, Implementation Challenges, and Limitations .....              | 20 |
| 6.3.1 Technical Limitations.....                                                   | 20 |
| 6.3.2 Implementation and Organisational Risks.....                                 | 20 |
| 6.3.3 Security and Ethical Considerations.....                                     | 21 |
| 6.4 Suggested Future Enhancements .....                                            | 21 |
| 7. Conclusion .....                                                                | 21 |

# 1. UNDERSTANDING THE PROBLEM STATEMENT

---

## 1.1 Problem Statement in Own Words

Flooding is one of the most frequent and destructive natural disasters faced by riverine and coastal regions, causing large-scale damage to human life, property, agriculture, and infrastructure. Traditional flood monitoring relies heavily on manually operated river gauges, periodic field surveys, and delayed satellite bulletins, which together produce a slow and fragmented picture of an evolving flood event. By the time officials collate readings from multiple sources and issue a warning, the water may have already reached vulnerable habitations, leaving little time for evacuation or relief mobilisation.

The core problem this case study addresses is the absence of a fast, automated, and reasonably accurate mechanism to detect flooded regions from remotely sensed imagery (satellite or aerial/drone photographs) and to convert that detection into an actionable, timely warning for disaster management authorities and the public. The problem is fundamentally one of image classification and semantic understanding under real-world constraints: images arrive continuously, must be processed with minimal latency, must work across varying terrain and lighting conditions, and must produce outputs that non-technical emergency responders can immediately act upon.

From a software engineering perspective, the problem is not simply "train a deep learning model" — it is the design of an end-to-end system that acquires imagery, runs inference using a chosen convolutional neural network (CNN) architecture, validates and stores results, and pushes alerts through a reliable software pipeline to stakeholders who may have limited connectivity or technical literacy.

## 1.2 Objectives of the Proposed System

- To automatically classify satellite or aerial images into flooded and non-flooded regions using deep learning models, reducing dependence on manual visual inspection.
- To compare and select the most suitable CNN architecture (e.g., EfficientNet-B1, Xception, VGG16, ResNet101) for the flood-classification task based on accuracy, inference speed, and computational cost.
- To design a scalable software pipeline that ingests imagery from multiple sources (satellite feeds, drones, crowd-sourced photographs) and processes them with minimal delay.

- To generate real-time, geotagged flood alerts and dashboards that district disaster management authorities can use for evacuation planning and resource allocation.
- To provide a maintainable, modular architecture that allows new models, new geographic regions, or new data sources to be added without redesigning the entire system.
- To ensure the system is usable by non-expert emergency personnel through a simple web/mobile interface rather than requiring data-science expertise.
- To incorporate reliability and security safeguards so that false alarms are minimised and the alerting channel cannot be tampered with or spoofed.

### **1.3 Scope of the Problem**

The scope of this case study is bounded to the design and analysis of a software system for image-based flood detection and alerting; it does not extend to the meteorological modelling of rainfall or the hydrological simulation of river discharge, though the system is designed to accept such data as supplementary inputs in future iterations.

- In-scope: acquisition of satellite/drone imagery, pre-processing, deep-learning-based classification of flooded vs non-flooded regions, comparative evaluation of CNN architectures, alert generation, dashboard visualisation, and role-based access for disaster-management stakeholders.
- In-scope: a software architecture that can be deployed regionally (e.g., district or state level) and scaled to a national level with additional compute and storage resources.
- Out-of-scope: physical sensor/hardware manufacturing, deployment of new satellites, rainfall-runoff hydrological modelling, and legal/regulatory frameworks for disaster compensation.
- Out-of-scope (for this iteration): fully autonomous drone flight control; the system assumes imagery is supplied by existing satellite programmes, government drone units, or citizen uploads.

The target users are state and district disaster management authorities, meteorological departments, NGOs involved in relief work, and the general public in flood-prone areas who receive alerts via SMS or a mobile application.

## 2. KEY STAKEHOLDERS AND CHALLENGES

---

### 2.1 Stakeholder Identification, Roles and Responsibilities

An effective flood detection and early-warning system depends on the coordinated participation of technical, governmental, and community stakeholders. The table below identifies each stakeholder group together with their principal responsibilities within the system.

| Stakeholder                                                                        | Roles And Responsibilities                                                                                                        | Key Challenges Faced                                                                                                                                                          |
|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Disaster Management Authority (State/District)</b>                              | Primary decision-maker; issues evacuation orders; monitors dashboard; allocates relief resources based on alerts.                 | Needs highly reliable, low-latency alerts; cannot act on false positives; must justify decisions to the public.                                                               |
| <b>Meteorological Department</b>                                                   | Supplies supplementary rainfall/weather data; validates flood predictions against independent weather models.                     | Data-format mismatches; needs system to integrate external APIs without manual re-entry.                                                                                      |
| <b>Remote Sensing / Satellite Data Providers (e.g., ISRO Bhuvan, Sentinel Hub)</b> | Supplies raw satellite imagery at scheduled intervals; defines image resolution and revisit frequency.                            | Revisit time is not always fast enough for rapidly evolving flash floods; cloud cover can obscure imagery.                                                                    |
| <b>Software Development Team</b>                                                   | Designs, builds, tests, and maintains the detection pipeline, model-serving layer, and dashboard/mobile application.              | Must keep the ML models updated as terrain and climate patterns shift; must maintain uptime during disaster events (when load is highest and infrastructure is most fragile). |
| <b>Data Scientists / ML Engineers</b>                                              | Train, validate, and benchmark CNN architectures (EfficientNet-B1, Xception, VGG16, ResNet101); tune models for regional terrain. | Limited labelled flood imagery for many regions; class imbalance between flooded and non-flooded samples; model drift over seasons.                                           |

|                                                        |                                                                                                               |                                                                                                                                   |
|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>Local Government / Panchayat Officials</b>          | Act as the last-mile relay of alerts to villages; coordinate ground-level evacuation and shelter setup.       | Poor connectivity in rural/flood-affected zones; may not trust automated alerts without a human-verified escalation step.         |
| <b>NGOs and Relief Organisations</b>                   | Use flood-extent maps to plan relief material distribution and rescue routes.                                 | Need exportable, standard-format data (e.g., GeoJSON) rather than a closed proprietary dashboard.                                 |
| <b>General Public / Residents in Flood-Prone Areas</b> | End recipients of SMS/app alerts; expected to evacuate or take precautions based on warnings.                 | Alert fatigue from false alarms; limited smartphone or internet access in remote regions; language and literacy barriers.         |
| <b>System Administrators / IT Operations</b>           | Maintain servers, cloud infrastructure, model-serving endpoints, and database backups; monitor system health. | Must guarantee high availability exactly when infrastructure (power, network) is most likely to be disrupted by the flood itself. |
| <b>Regulatory / Government IT Security Bodies</b>      | Audit the system for data protection compliance and cybersecurity standards before public deployment.         | Balancing rapid deployment during emergencies against rigorous security review cycles.                                            |

## 2.2 Major Challenges and Issues Faced by Stakeholders

### 2.2.1 Data-Related Challenges

- Scarcity of high-quality, labelled flood-imagery datasets for many regions, forcing reliance on transfer learning from generic datasets that may not represent local terrain.
- Class imbalance — non-flooded imagery vastly outnumbers flooded imagery in most archives, biasing naive models toward under-predicting floods.
- Cloud cover and nighttime conditions reduce the usability of optical satellite imagery, creating gaps exactly when floods are most active (during monsoon storms).

### **2.2.2 Operational and Organisational Challenges**

- Coordination overhead between the meteorological department, satellite data providers, and the disaster management authority, each of which may use different data formats and update schedules.
- Trust deficit: local officials and residents may be reluctant to act on an automated alert without a human-verified confirmation step, especially early in the system's deployment.
- Last-mile communication gaps in rural or hilly terrain where mobile network coverage is itself disrupted by the flood.

### **2.2.3 Technical Challenges**

- Latency: satellite revisit intervals and heavy CNN inference times can delay detection beyond the window needed for effective evacuation, particularly for flash floods.
- Scalability: the system must handle sudden spikes in image volume and dashboard usage during an actual flood event, when the infrastructure is under the most stress.
- Model maintenance: seasonal changes in vegetation, urban growth, and terrain modification require the CNN models to be periodically retrained, which demands an ongoing MLOps process rather than a one-time deployment.
- Security: alert channels must be protected from spoofing or tampering, since a fabricated 'all-clear' or false alarm message could itself cause serious harm.

### **3. ANALYSIS OF THE CURRENT APPROACH**

---

#### **3.1 Description of the Existing System / Current Process**

The prevailing flood-monitoring process in most regions is a semi-manual, multi-agency workflow. River-gauge stations report water levels at fixed intervals (often several hours apart) to a central hydrology office. Meteorological departments separately issue rainfall forecasts and heavy-rainfall warnings. When available, satellite imagery is manually reviewed by trained analysts at remote-sensing agencies, who visually compare recent images with historical baselines to estimate flood extent — a process that can take many hours to a day per region.

These independent data streams are then manually compiled, often over phone calls, emails, or physically distributed bulletins, into a situation report that is passed to the district disaster management authority. The authority uses this report, combined with local ground reports from panchayat officials, to decide whether to issue evacuation orders. Public alerts are typically broadcast through television, radio, loudspeaker announcements, and, in some regions, bulk SMS services operated with limited targeting (i.e., broadcast to an entire district rather than the specific flooded sub-region).

#### **3.2 Strengths of the Current System**

- Human expert review of satellite imagery brings contextual judgement (e.g., distinguishing genuine floodwater from shadows, water bodies, or agricultural fields under irrigation) that automated systems can initially struggle with.
- Established institutional relationships between the meteorological department, hydrology offices, and disaster management authorities already exist and function reasonably well for slow-onset riverine floods.
- Ground-level networks of panchayat officials and NGOs provide valuable local knowledge that purely remote, image-based systems cannot capture on their own.
- Redundant communication channels (radio, television, loudspeaker) reach even areas without internet or mobile connectivity.

### 3.3 Limitations of the Current System

- High latency: manual image review and multi-agency compilation can take many hours, which is unacceptably slow for flash floods that develop within a few hours of intense rainfall.
- Limited scalability: a small team of remote-sensing analysts cannot simultaneously review imagery for every flood-prone region during a widespread event, such as a multi-district monsoon flood.
- Inconsistent alerting granularity: broadcast SMS and radio alerts are often issued at the district level, causing unaffected residents to receive unnecessary warnings (alert fatigue) while genuinely affected pockets may not be pinpointed quickly enough.
- No standardised digital dashboard: situation reports are frequently compiled via phone calls and documents rather than a shared, real-time digital system, making it difficult for NGOs and relief teams to access a single source of truth.
- Dependence on individual expertise: the quality of manual satellite-image interpretation varies with analyst experience, and there is no systematic, repeatable benchmark of accuracy.

### 3.4 Gaps That Need to Be Addressed

- A gap between data acquisition and actionable alerting — raw imagery exists, but the interpretation-to-alert pipeline is slow and manual.
- A gap in automation — no consistent, quantifiable, and repeatable model is used to classify flooded regions, making outcomes analyst-dependent rather than system-dependent.
- A gap in geographic precision — alerts are not fine-grained enough to avoid unnecessary panic in unaffected areas while ensuring affected pockets are not missed.
- A gap in integration — meteorological, satellite, and ground-report data streams are not unified into a single software platform accessible to all stakeholders simultaneously.

These gaps collectively motivate a software-engineered solution that automates image interpretation using deep learning, integrates multiple data sources into a single pipeline, and delivers precise, low-latency, role-based alerts — while retaining a human-in-the-loop verification step to preserve the contextual judgement that the current manual process provides.

## 4. PROPOSED SOFTWARE SOLUTION USING SOFTWARE ENGINEERING PRINCIPLES

---

### 4.1 Overview of the Proposed System

The proposed solution is a modular, cloud-deployable software platform — referred to here as the Flood Detection and Early Warning System (FDEWS) — that automates the classification of satellite and aerial imagery into flooded and non-flooded regions using a deep convolutional neural network, and converts these classifications into precise, role-based alerts for disaster management stakeholders. The system is designed as a layered architecture comprising a data-ingestion layer, a machine-learning inference layer, an alerting and geospatial layer, and a presentation layer (dashboard and mobile application), connected through well-defined REST APIs and a message queue for asynchronous processing.

Rather than replacing human expertise, FDEWS is designed to augment it: automated classification handles the volume and speed of image interpretation, while a human-in-the-loop verification step preserves the contextual judgement that experienced analysts currently provide, directly addressing the trust and false-alarm concerns identified in the stakeholder analysis.

### 4.2 Major Functional Modules

| Module                                        | Description                                                                                                                                                                                                                |
|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Data Ingestion Module</b>                  | Collects satellite imagery (e.g., Sentinel-1/2, Bhuvan feeds), drone/UAV photographs, and citizen-uploaded images via API connectors and a mobile upload interface; normalises formats and metadata (geotags, timestamps). |
| <b>Pre-processing Module</b>                  | Performs image cleaning, cloud-mask filtering, resizing, normalisation, and augmentation before the imagery is passed to the inference engine.                                                                             |
| <b>Model Inference Engine</b>                 | Hosts the trained CNN model (selected from EfficientNet-B1, Xception, VGG16, or ResNet101 after comparative evaluation) and returns a flooded/non-flooded classification with a confidence score for each image tile.      |
| <b>Model Training &amp; Evaluation Module</b> | Offline pipeline used by data scientists to retrain models on newly labelled data, benchmark architectures, and version-control model artefacts.                                                                           |

|                                                   |                                                                                                                                                                                                 |
|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Geo-Tagging &amp; Mapping Module</b>           | Overlays classification results on a geographic information system (GIS) map, converting raw predictions into flood-extent polygons for each monitored region.                                  |
| <b>Alert Generation &amp; Notification Module</b> | Applies threshold and confidence rules to decide when an alert should be raised; dispatches SMS, push notifications, and dashboard alerts to relevant stakeholders based on role and geography. |
| <b>Human-in-the-Loop Verification Module</b>      | Routes high-confidence flood detections to a duty officer for a quick confirmation before a public-facing alert is broadcast, preserving expert oversight.                                      |
| <b>Dashboard &amp; Visualisation Module</b>       | Web-based dashboard for disaster management authorities and NGOs showing live flood maps, historical trends, and system health metrics.                                                         |
| <b>Mobile Application Module</b>                  | Lightweight app for panchayat officials and residents to receive alerts, view local flood maps, and report ground conditions (crowd-sourced verification).                                      |
| <b>User &amp; Access Management Module</b>        | Role-based authentication and authorisation for different stakeholder categories (administrator, analyst, authority, public).                                                                   |
| <b>Logging, Audit &amp; Reporting Module</b>      | Maintains an immutable log of all alerts issued, model versions used, and manual overrides, supporting post-event audits and continuous improvement.                                            |

### 4.3 Functional Requirements

| ID          | Requirement                                                                                                                                        |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>FR-1</b> | The system shall ingest satellite and drone imagery from configured data sources at scheduled intervals.                                           |
| <b>FR-2</b> | The system shall pre-process incoming images (resizing, normalisation, cloud-mask filtering) before inference.                                     |
| <b>FR-3</b> | The system shall classify each image or image tile as 'flooded' or 'non-flooded' with an associated confidence score using the selected CNN model. |
| <b>FR-4</b> | The system shall allow data scientists to switch between or retrain supported CNN architectures without redeploying the entire application.        |

|              |                                                                                                                                       |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>FR-5</b>  | The system shall generate geo-referenced flood-extent maps from classification outputs.                                               |
| <b>FR-6</b>  | The system shall route high-confidence flood detections to a human duty officer for verification before public alerts are dispatched. |
| <b>FR-7</b>  | The system shall send targeted SMS/app notifications to stakeholders within the affected geographic boundary only.                    |
| <b>FR-8</b>  | The system shall provide a web dashboard displaying live flood maps, alert history, and model performance metrics.                    |
| <b>FR-9</b>  | The system shall allow citizens to upload geotagged photographs as supplementary crowd-sourced evidence.                              |
| <b>FR-10</b> | The system shall maintain an audit log of every alert issued, including model version, confidence score, and verifying officer.       |

## 4.4 Non-Functional Requirements

| Quality Attribute                 | Requirement                                                                                                                                                                                    |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Performance</b>                | The end-to-end pipeline (ingestion to alert dispatch) shall complete within a defined target latency (e.g., under 15 minutes) for a standard image batch, so alerts remain actionable.         |
| <b>Scalability</b>                | The system shall handle a sudden surge in imagery and dashboard traffic during a large-scale flood event without significant degradation, using horizontally scalable, containerised services. |
| <b>Reliability / Availability</b> | The system shall target high availability (e.g., 99.5%+) during active flood seasons, with redundant servers and failover for the alerting pipeline.                                           |
| <b>Maintainability</b>            | The modular architecture shall allow models, data sources, or notification channels to be updated independently with minimal regression risk.                                                  |
| <b>Usability</b>                  | The dashboard and mobile app shall be usable by non-technical disaster-management staff with minimal training, using simple visual cues (colour-coded flood-risk levels).                      |
| <b>Security</b>                   | All data transmissions shall be encrypted in transit; role-based access control shall restrict who can issue public alerts; audit logs shall be tamper-evident.                                |
| <b>Interoperability</b>           | The system shall expose standard data formats (GeoJSON, REST APIs) so NGOs and third-party disaster-response tools can consume flood-extent data.                                              |

|                    |                                                                                                                                                                                |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Portability</b> | The core inference and alerting services shall be containerised so the system can be deployed on-premises, on government cloud, or on public cloud infrastructure as required. |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 4.5 Incorporation of Software Engineering Principles

### 4.5.1 Modularity

The system decomposes cleanly into independent modules — ingestion, pre-processing, inference, geo-tagging, alerting, verification, dashboard, and mobile app — each exposed through a well-defined API contract. This allows, for example, the Model Inference Engine to be upgraded from VGG16 to EfficientNet-B1 without touching the alerting or dashboard modules, directly supporting the comparative-architecture evaluation goal of the case study.

### 4.5.2 Scalability

Compute-intensive components (image pre-processing and CNN inference) are containerised using Docker and orchestrated with Kubernetes, allowing horizontal auto-scaling of inference workers when image volume spikes during a widespread flood event. A message queue (e.g., Kafka or RabbitMQ) decouples ingestion from inference so that bursts of incoming imagery do not overwhelm the model-serving layer.

### 4.5.3 Maintainability

A layered architecture, consistent API versioning, automated test suites, and a CI/CD pipeline allow developers to make isolated changes (e.g., add a new satellite data source or a new notification channel) with automated regression testing before deployment, reducing the risk of introducing defects into a live, safety-critical system.

### 4.5.4 Reliability

Redundant inference nodes, database replication, and a fallback alerting channel (e.g., IVR/voice calls when SMS delivery fails) ensure that a single point of failure does not silence the alerting pipeline during the exact period when the system is most needed. Health-check monitoring and automated failover further support this requirement.

#### **4.5.5 Usability**

The dashboard uses simple colour-coded flood-risk indicators, GIS-based map overlays, and minimal-click workflows so that disaster-management personnel with limited technical background can interpret system output quickly under time pressure. The mobile application is designed for low-bandwidth conditions typical of rural, flood-affected areas.

#### **4.5.6 Security**

Role-based access control restricts who can view raw data versus who can authorise a public alert; all API traffic is encrypted using TLS; alert messages are digitally signed so that recipients (and downstream systems) can verify authenticity and prevent spoofed 'false all-clear' or panic-inducing messages; audit logs are stored in an append-only, tamper-evident format.

### **4.6 High-Level Architecture Summary**

Data flows from the ingestion layer, through pre-processing and the CNN-based inference engine, into a geospatial processing component that produces flood-extent polygons. These results pass through a rules engine that determines alert thresholds, which then routes high-confidence detections to a human verification step before dispatch through the notification module to SMS gateways, push-notification services, and the web dashboard. This pipeline is orchestrated asynchronously via a message queue, ensuring that a slow step (e.g., large-image inference) does not block the ingestion of new imagery.

## 5. JUSTIFICATION OF THE CHOSEN METHODOLOGY/Framework

---

### 5.1 Selected Methodology: Agile (Scrum) combined with DevOps Practices

This case study recommends the Agile Scrum framework, augmented with DevOps practices for continuous integration and deployment of the machine-learning components, as the most suitable software development approach for the Flood Detection and Early Warning System.

### 5.2 Comparison of Candidate Methodologies

| Model                                | Characteristics                                                                                                                                                   | Suitability Assessment                                                                                                                                                   |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Waterfall                            | Sequential, rigid phases; poor fit when requirements (e.g., which CNN architecture performs best) are not known upfront.                                          | Not selected — the case study explicitly requires iterative comparison of multiple deep-learning models, which Waterfall's rigid phase-gating does not accommodate well. |
| Spiral                               | Risk-driven, good for high-risk projects but heavyweight in documentation and management overhead.                                                                | Considered but rejected as overly heavyweight for a system that benefits from frequent, lightweight iterations rather than formal risk-review cycles at every stage.     |
| Agile (Scrum)                        | Iterative sprints, continuous stakeholder feedback, incremental delivery of working modules; naturally supports experimentation with multiple ML models.          | Selected as the primary framework.                                                                                                                                       |
| DevOps (as a complementary practice) | Continuous integration/continuous deployment, automated testing, and monitoring; complements Agile for a system requiring frequent model updates and high uptime. | Adopted alongside Scrum for the ML-serving and deployment pipeline.                                                                                                      |

### 5.3 Justification for Selecting Agile Scrum

- Evolving and experimental requirements: since the project explicitly involves comparing multiple CNN architectures (EfficientNet-B1, Xception, VGG16, ResNet101), the optimal model cannot be finalised at the start of the project. Scrum's iterative sprints allow the team to experiment, benchmark, and refine model choice across successive iterations rather than committing upfront, as Waterfall would require.
- Incremental stakeholder feedback: disaster-management authorities, NGOs, and local officials can review working increments (e.g., a basic dashboard, then geo-tagged alerts, then role-based notifications) every sprint, ensuring the system evolves to match real operational needs rather than a static specification.
- Risk management through short cycles: because this is a safety-critical system, catching usability or accuracy issues early — within a two-to-three week sprint — is far less costly than discovering them after a full Waterfall-style development cycle.
- Cross-functional collaboration: Scrum's structure (daily stand-ups, sprint reviews, retrospectives) naturally supports the tight collaboration needed between data scientists (model training), backend developers (pipeline and API), and domain experts (disaster-management representatives) throughout development.
- Support for MLOps: pairing Scrum with DevOps/CI-CD pipelines allows retrained models to be automatically tested, containerised, and deployed to production on a regular cadence, which is essential because flood-detection models require periodic retraining as terrain, urbanisation, and climate patterns shift over time.

### 5.4 How the Chosen Framework Supports Successful Development and Maintenance

Agile Scrum organises development into fixed-length sprints (commonly two to three weeks), each producing a potentially shippable increment. For FDEWS, an illustrative sprint roadmap would begin with the data-ingestion and pre-processing modules, followed by sprints dedicated to training and benchmarking the four candidate CNN architectures, then sprints for the alerting and geo-tagging modules, and finally sprints for the dashboard, mobile application, and security hardening.

Sprint reviews with disaster-management stakeholders after each increment ensure that features such as alert granularity, dashboard usability, and notification channels are validated against real operational needs before the next sprint begins. Sprint retrospectives allow the development team to continuously refine its own estimation and technical practices. DevOps automation — continuous integration pipelines that run automated tests on every code change, and continuous deployment to a staging environment — ensures that as new, better-performing models are identified through ongoing benchmarking, they can be safely rolled into production with minimal manual effort and reduced regression risk, supporting long-term maintainability well beyond the initial release.

## 6. EXPECTED OUTCOMES AND LIMITATIONS

---

### 6.1 Expected Benefits of the Proposed Solution

- Significantly reduced detection-to-alert latency compared with the manual, multi-agency compilation process, improving the effective evacuation window for at-risk residents.
- Consistent, repeatable, and benchmarked classification accuracy across regions, rather than accuracy that varies with individual analyst experience.
- Finer geographic precision in alerting, reducing unnecessary alerts to unaffected residents while ensuring genuinely affected pockets are not missed.
- A single, shared digital source of truth (the dashboard) accessible to disaster-management authorities, NGOs, and relief organisations simultaneously, improving coordination during an active flood event.
- A scalable platform capable of extending to additional regions or additional data sources (e.g., new satellite constellations, additional drone fleets) without a full system redesign.
- A structured, auditable record of every alert and the model/version behind it, supporting post-event review and continuous system improvement.

### 6.2 How the Solution Addresses the Identified Challenges

- Data scarcity and class imbalance are addressed through transfer learning on pre-trained CNN backbones (EfficientNet-B1, Xception, VGG16, ResNet101) and targeted data augmentation, reducing the amount of region-specific labelled data required.
- The trust deficit identified among local officials and residents is addressed through the human-in-the-loop verification module, ensuring automated detections are confirmed by a duty officer before public dispatch.
- Coordination overhead between meteorological, satellite, and ground-report sources is addressed by the unified data-ingestion and dashboard layers, replacing ad hoc phone/email compilation with a shared digital platform.

- Latency concerns are addressed through the asynchronous, queue-based pipeline and horizontally scalable inference layer, reducing the time between image acquisition and alert dispatch.
- Last-mile communication gaps are partially addressed through multi-channel notification (SMS, push notification, and continued support for existing radio/loudspeaker channels), rather than relying solely on the mobile application.

## **6.3 Potential Risks, Implementation Challenges, and Limitations**

### **6.3.1 Technical Limitations**

- Model accuracy remains fundamentally dependent on the quality, resolution, and recency of available satellite/drone imagery; heavy cloud cover during intense storms can degrade optical-imagery-based detection, which may require future integration of radar-based (SAR) imagery that can penetrate cloud cover.
- False positives or false negatives, even at a low rate, can either cause alert fatigue or, more seriously, fail to warn an affected population; achieving the confidence-threshold balance appropriate for a safety-critical system requires careful, ongoing calibration.
- Model drift over time (due to urbanisation, seasonal vegetation change, or new construction) requires a sustained MLOps discipline; without it, accuracy could silently degrade after initial deployment.

### **6.3.2 Implementation and Organisational Risks**

- Integration risk with legacy government IT systems and data-sharing agreements between meteorological, remote-sensing, and disaster-management agencies may take longer to negotiate than the technical development itself.
- Connectivity limitations in the most flood-vulnerable rural areas may limit the reach of the mobile application and SMS-based alerts, requiring continued reliance on traditional broadcast channels as a fallback.
- Budget and infrastructure constraints at the district level may limit the pace at which the system can be rolled out beyond an initial pilot region.

### 6.3.3 Security and Ethical Considerations

- A compromised or spoofed alerting channel could cause serious public harm (panic from a false alarm, or complacency from a suppressed real alert), making the security hardening of the notification pipeline a critical, non-negotiable requirement rather than an optional enhancement.
- Data privacy considerations arise from citizen-uploaded, geotagged photographs, requiring clear data-retention and consent policies.

### 6.4 Suggested Future Enhancements

- Integration of Synthetic Aperture Radar (SAR) imagery to maintain detection capability under cloud cover and at night, complementing the optical-imagery-based CNN pipeline.
- Fusion of the image-based classification with hydrological and rainfall-runoff models for improved lead time on flash floods, moving from a purely image-reactive system toward a predictive one.
- Incorporation of an ensemble of the benchmarked CNN architectures (e.g., combining EfficientNet-B1 and ResNet101 predictions) to improve robustness beyond any single model.
- Expansion of the crowd-sourced verification module with lightweight on-device image validation to reduce reliance on network connectivity in the most remote areas.
- Deeper integration with national emergency-alert broadcast systems and multilingual voice-call (IVR) alerts to further close last-mile communication gaps.
- Long-term establishment of a continuous model-retraining pipeline (MLOps) that automatically incorporates newly verified flood/non-flood imagery to counter model drift.

## 7. CONCLUSION

---

This case study analysed the problem of slow, manual, multi-agency flood monitoring and proposed a modular, deep-learning-based Flood Detection and Early Warning System that automates image classification using benchmarked CNN architectures, while preserving essential human oversight through a verification step. By applying core software engineering principles — modularity, scalability, maintainability, reliability, usability, and security — and adopting an Agile Scrum methodology supported by DevOps practices, the proposed solution offers a practical, incrementally deliverable path

to significantly reducing flood-alert latency and improving coordination among disaster-management stakeholders, while candidly acknowledging the technical, organisational, and ethical limitations that must be managed throughout implementation.